

Lab Session 10

Software Testing - Manual Testing

SOFTWARE TESTING

Software testing is an important activity in the **software development lifecycle (SDLC)** that ensures the software application works correctly and meets the specified requirements. The purpose of software testing is to identify defects, verify functionality, improve reliability, and ensure that the system behaves as expected under different conditions.

Testing plays a critical role in delivering **high-quality software products**. Without proper testing, software systems may contain bugs that could cause system failures, incorrect outputs, security vulnerabilities, or poor user experience.

Software testing helps to:

- Detect defects before deployment
- Verify that requirements are correctly implemented
- Improve software reliability and stability
- Enhance system performance
- Ensure a better user experience

Software testing can be broadly categorized into two major approaches:

- **Manual Testing**
- **Automation Testing**

Both approaches are widely used in software development depending on project requirements.

AUTOMATION TESTING

Automation testing uses specialized tools and scripts to automatically execute test cases. Instead of manually performing each test step, testers create automated scripts that simulate user actions and verify expected outcomes.

Automation testing is particularly useful for **repetitive and large-scale testing tasks**. It is commonly used for:

- Regression testing
- Performance testing
- Load testing
- Continuous integration testing

Automation tools commonly used in the industry include:

- Selenium
- Cypress
- TestComplete
- Appium

Although automation testing provides faster execution and improved efficiency, it requires initial setup, programming knowledge, and maintenance of test scripts.

MANUAL TESTING

Manual testing is the process of testing software **manually without using automation tools**. In this method, testers interact with the application just like real users and verify whether the system behaves according to the expected requirements.

Manual testing allows testers to explore the application and detect issues related to usability, interface design, and unexpected behaviors that automated tests might miss.

Manual testing is commonly used for:

- Exploratory testing
- Usability testing
- Interface testing
- Ad-hoc testing

Even in projects where automation testing is implemented, manual testing remains essential for validating **user experience and complex user scenarios**.

STEPS FOR MANUAL TESTING

Manual testing generally follows a structured process to ensure systematic verification of software functionality.

1. Define the Test Scenario

A test scenario describes a **high-level functionality** that needs to be tested.

Example:

Test Scenario:

Verify the login functionality of the **Facebook login page**.

This scenario defines the feature being tested without going into detailed steps.

2. Create Test Case ID

Each test case must have a **unique identifier** for tracking and documentation purposes.

Example:

- TC-001
- TC-002
- TC-003

Unique IDs help testers easily reference and manage test cases.

3. Test Case Description

The description explains **what the test case verifies**.

Example:

Verify that a user can successfully log in using valid credentials.

4. Preconditions

Preconditions are the conditions that must be satisfied before executing the test.

Examples:

- User must have a registered Facebook account
- Internet connection must be active
- User must be on the login page

5. Test Steps

Test steps define the **exact sequence of actions** the tester must perform.

Example steps:

1. Open Facebook login page
2. Enter valid username
3. Enter valid password
4. Click the **Login** button

6. Expected Result

The expected result describes what the system **should do** if the test passes.

Example:

User should be successfully logged in and redirected to the homepage.

7. Actual Result

The actual result is the **observed system behavior** after executing the test.

Example:

User logged in successfully and homepage displayed.

8. Status

Based on the comparison of **expected vs actual result**, the test case is marked as:

- **Pass**
- **Fail**

9. Remarks

Remarks include additional comments or observations made during testing.

Example:

Login fails when password contains special characters.

TEST SCENARIO

Scenario: Verify the Login Functionality of Facebook Login Page

The login page allows users to access their accounts by entering valid credentials.

The following functionalities should be verified:

- Login with valid credentials
- Login with incorrect password
- Login with invalid username
- Login with empty fields

TEST CASE DOCUMENTATION FORMAT

Before writing test cases, detailed information about the test should be recorded.

Test Case Details

Test Case Name: Login

Created By: _____

Executed By: _____

Date Created: _____

Execution Date: _____

Application Name: Facebook

Test Case Table

Test ID	Test Name	Test Description	Precondition	Test Steps	Expected Result	Actual Result	Post Condition	Status
TC-01	Login with Valid Credentials	Verify that user can login with valid username and password	User must have a registered Facebook account	1. Open Facebook login page 2. Enter valid username 3. Enter valid password 4. Click Login button	User should be redirected to homepage after successful login	User redirected to homepage	User session starts	Pass
TC-02	Login with Invalid Password	Verify system behavior when user enters incorrect password	User must have a valid account	1. Open login page 2. Enter valid username 3. Enter incorrect password 4. Click Login	System should display error message "Incorrect password"	"Incorrect password"	User remains on login page	Pass
TC-03	Login with Invalid Username	Verify behavior when username does not exist	Internet connection active	1. Open login page 2. Enter invalid username 3. Enter password 4. Click Login	System should display error message "User not found"	User not found	Login denied	Pass
TC-04	Login with Empty Fields	Verify that system prevents login when fields are empty	User on login page	1. Open login page 2. Leave username and password empty 3. Click Login	Validation message should appear	Validation message appear	Login prevented	Pass

TC-05	Login with Username Only	Verify behavior when password field is empty	User on login page	1. Enter username 2. Leave password empty 3. Click Login	System should show validation error	Validation error	Login denied	Pass
TC-06	Login with Password Only	Verify behavior when username field is empty	User on login page	1. Leave username empty 2. Enter password 3. Click Login	Validation message displayed	Validation message	Login denied	Pass

EXERCISE

Task 1

Write code and prepare test cases for a function that **returns the largest of three real numbers**.

The function accepts three numbers as input and returns the largest value among them.

Students should test scenarios such as all three numbers are different, two numbers are equal, Negative numbers etc

Prepare **at least 6 test cases** that verify the correctness of the function.

Task 2

Write code and Design test cases for a function that **calculates the Least Common Multiple (LCM) of two positive integers**.

The function takes two positive integers and returns their least common multiple.

Students should consider cases such as:

- Two different numbers
- Two identical numbers
- Prime numbers
- One number larger than the other
- Edge cases with small values

Prepare **at least 6 test cases** to verify the correctness of the function.

Task 3

Design test cases for a function that **calculates the square root of a real number**.

The function receives a real number as input and returns its square root.

Students should analyze different situations such as Positive numbers, Invalid inputs etc

Prepare **at least 6 test cases** for this function.

Task 4

Write code and prepare test cases for a function that **sorts an array of integers in ascending order**.

The function receives:

- An array **A**
- A variable **n**, which represents the number of elements in the array

The function returns the array sorted in ascending order.

Prepare **at least 6 test cases** verifying that the sorting operation works correctly.

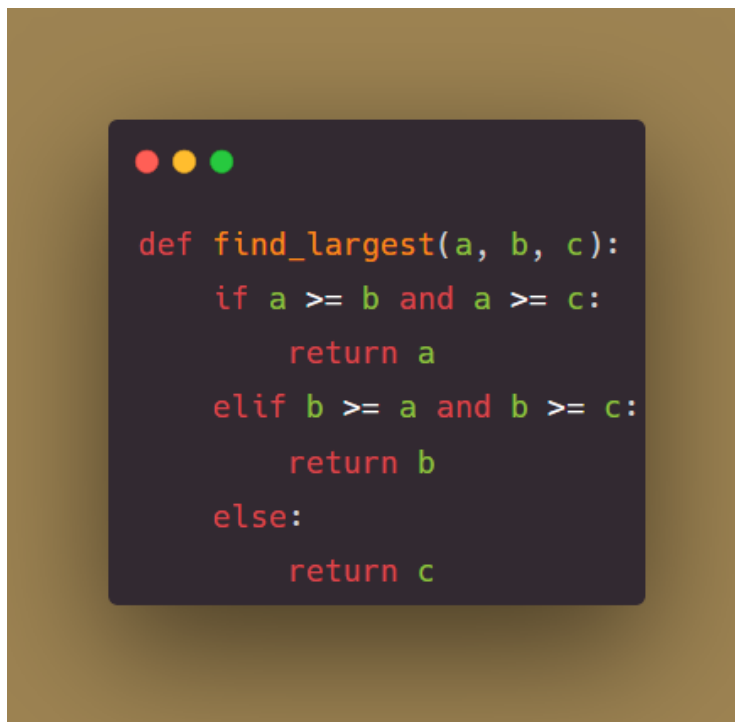
Task 1

Write code and prepare test cases for a function that **returns the largest of three real numbers**.

The function accepts three numbers as input and returns the largest value among them.

Students should test scenarios such as all three numbers are different, two numbers are equal, Negative numbers etc

Prepare **at least 6 test cases** that verify the correctness of the function.



A	B	C	D	E	F
Test ID	Test Description	Input (a, b, c)	Expected Result	Actual Result	Status
TC-01	All numbers are different and positive	10, 20, 15	20	20	Pass
TC-02	Two numbers are equal	5, 5, 2002	5	5	Pass
TC-03	All numbers are identical	7, 7, 2007	7	7	Pass
TC-04	Includes negative numbers	-10, -5, -20	-5	-5	Pass
TC-05	Mix of positive and negative numbers	-1, 0, 1	1	1	Pass
TC-06	Floating point (real) numbers	2.5, 2.1, 2.9	2.9	2.9	Pass

Task 2

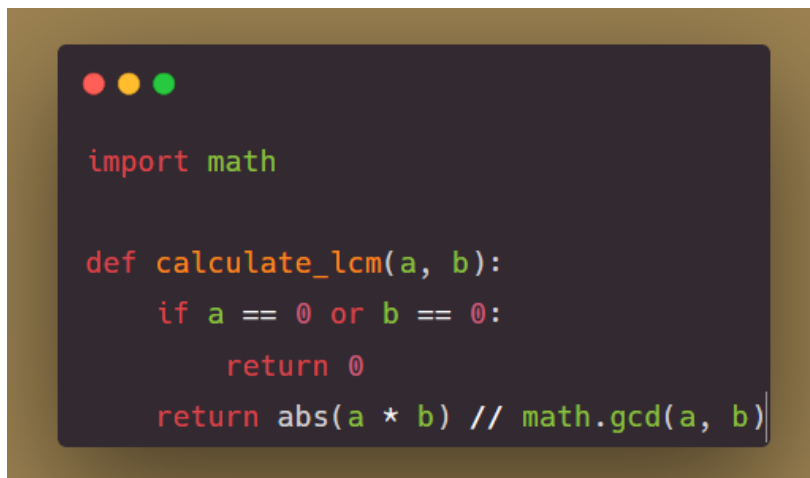
Write code and Design test cases for a function that **calculates the Least Common Multiple (LCM) of two positive integers**.

The function takes two positive integers and returns their least common multiple.

Students should consider cases such as:

- Two different numbers
- Two identical numbers
- Prime numbers
- One number larger than the other
- Edge cases with small values

Prepare **at least 6 test cases** to verify the correctness of the function.

A code editor window with a dark background and a light blue border. It contains Python code for calculating the LCM of two numbers. The code is as follows:

```
import math

def calculate_lcm(a, b):
    if a == 0 or b == 0:
        return 0
    return abs(a * b) // math.gcd(a, b)
```

Test ID	Test Description	Input (a, b)	Expected Result	Actual Result	Status
TC-07	Two different numbers	4, 6	12	12	Pass
TC-08	Two identical numbers	10, 10	10	10	Pass
TC-09	Two prime numbers	3, 7	21	21	Pass
TC-10	One number is a multiple of the other	2, 8	8	8	Pass
TC-11	Edge case with small values	1, 2	2	2	Pass
TC-12	Large positive integers	12, 15	60	60	Pass

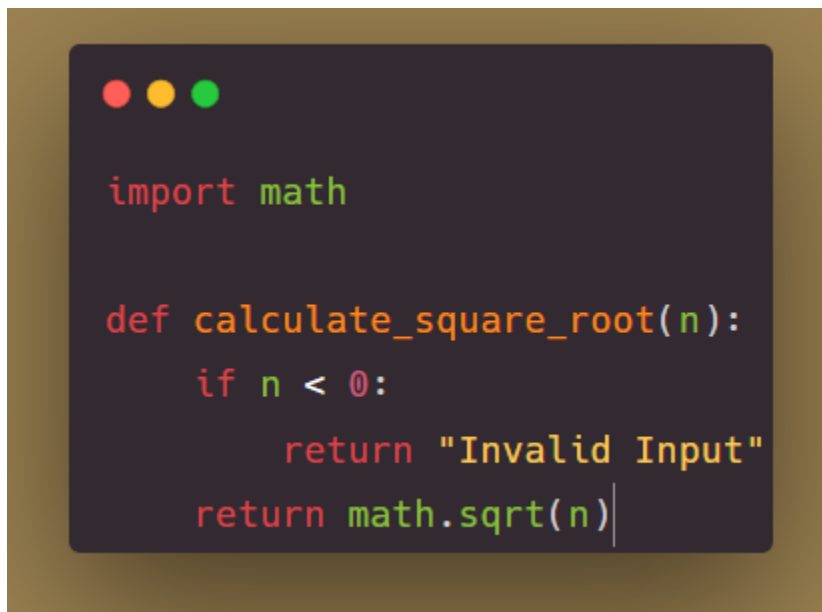
Task 3

Design test cases for a function that **calculates the square root of a real number**.

The function receives a real number as input and returns its square root.

Students should analyze different situations such as Positive numbers, Invalid inputs etc

Prepare **at least 6 test cases** for this function.



```
import math

def calculate_square_root(n):
    if n < 0:
        return "Invalid Input"
    return math.sqrt(n)
```

Test ID	Test Description	Input (n)	Expected Result	Actual Result	Status
TC-13	Perfect square (positive number)	16	4	4	Pass
TC-14	Non-perfect square	2	1.414...	1.414...	Pass
TC-15	Square root of zero	0	0	0	Pass
TC-16	Invalid input (negative number)	-9	"Invalid Input"	"Invalid Input"	Pass
TC-17	Large positive real number	10000	100	100	Pass
TC-18	Floating point input	0.25	0.5	0.5	Pass

Task 4

Write code and prepare test cases for a function that **sorts an array of integers in ascending order**.

The function receives:

- An array **A**
- A variable **n**, which represents the number of elements in the array

The function returns the array sorted in ascending order.

Prepare **at least 6 test cases** verifying that the sorting operation works correctly.

```
def sort_array(A, n):  
    # Slice to ensure we only sort up to n elements if the array is larger  
    return sorted(A[:n])
```

Test ID	Test Description	Input (A, n)	Expected Result	Actual Result	Status
TC-19	Unsorted positive integers	[5, 2, 9, 1], 4	[1, 2, 5, 9]	[1, 2, 5, 9]	Pass
TC-20	Already sorted array	[1, 2, 3], 3	[1, 2, 3]	[1, 2, 3]	Pass
TC-21	Reverse sorted array	[10, 5, 0], 3	[0, 5, 10]	[0, 5, 10]	Pass
TC-22	Array with identical elements	[4, 4, 4], 3	[4, 4, 4]	[4, 4, 4]	Pass
TC-23	Array with negative numbers	[-3, 0, -1, 5], 4	[-3, -1, 0, 5]	[-3, -1, 0, 5]	Pass
TC-24	Single element array	[100], 1	[100]	[100]	Pass